

TP Jour 3 — Réplication & haute disponibilité (4 h)

Module MIA4 · Conception et intégration d'un SGBD NoSQL — IPSSI Montpellier Jour 3 / 5 — Replica Set, élection, oplog, Write/Read Concern, résilience applicative

Le sharding est traité demain, en première demi-journée du Jour 4, sur un cluster déjà assemblé.

Aujourd'hui, une seule question vous occupe : **comment un service survit-il à la mort d'une machine ?**

Contexte professionnel

Vous êtes **ingénieur·e de production (SRE)** chez un éditeur qui exploite un **référentiel géodémographique américain** : la collection `census.zips` contient **29 470 codes postaux réels** des États-Unis (population, ville, État, coordonnées) — jeu de données officiel MongoDB `sample_training/zips`, issu du recensement US.

Aujourd'hui la base tourne sur **un seul mongod**, et la DSI impose un **SLA de 99,9 %** : plus aucune interruption de service acceptable lors d'une panne serveur. Ce chiffre autorise **43 minutes d'indisponibilité par mois** — pas une de plus.

Votre mission de la journée : **monter un Replica Set, provoquer des pannes et prouver chronomètre en main que le service survit**, puis **brancher une vraie application dessus et mesurer ce qu'elle perd pendant la bascule**. À 17 h, vous devrez annoncer un chiffre à la DSI, et le défendre.

Attention — ce TP se mesure, il ne se récite pas. Chaque réponse doit contenir la **commande exacte**, la **sortie observée** (copiée, pas paraphrasée) et, quand c'est demandé, un **temps mesuré sur votre machine**. Les délais que vous relèverez ne seront identiques à ceux de personne d'autre : c'est le but. Un correcteur rejouera vos commandes.

Objectifs pédagogiques (rappel du cours)

- Déployer un **Replica Set 3 nœuds** et lire son état (`rs.status()`, `rs.conf()`, `db.hello()`).
- Comprendre l'**oplog** : collection capped, idempotence, rôle dans la resynchronisation, dimensionnement.
- Provoquer et **chronométrer un failover**, distinguer arrêt propre et panne brutale, comprendre `electionTimeoutMillis` et le **quorum majoritaire**.
- Régler les curseurs **Write Concern / Read Concern / Read Preference** et constater leurs erreurs réelles.
- **Brancher une application** (PyMongo) sur le set et mesurer sa résilience réelle : découverte de topologie, `retryWrites`, écritures perdues.

Pré-requis techniques

- Docker Desktop **démarré**, au moins **3 Go de RAM** allouables (3 conteneurs MongoDB, 4 avec le bonus B1).

- Les fichiers fournis : `docker-compose.rs.yml`, `init-rs.js`, `watch_primary.py`, `writer.py`.
- Python 3.10+ avec `pymongo>=4.6` (Partie 5).

Livrables (Teams, avant 17h00)

1. `reponses_jour3.md` — commande + sortie observée pour chaque question, et R1→R4 rédigées.
2. `failover.md` — vos **mesures de bascule** (tableau : type de panne, délai, nœud élu).
3. `resilience.md` — la sortie **brute et horodatée** de votre `writer.py` pendant la panne, et votre décompte des écritures perdues.
4. `writer.py` complété, et les fichiers d'infra que vous avez réellement exécutés.

Partie 0 — Monter le Replica Set (≈ 25 min)

0.1 Libérer le port 27017

À faire en premier. La stack du Jour 1 occupe le port **27017**, celui dont le Replica Set a besoin. Si vous l'oubliez, `docker compose up` échouera sur un message de port déjà alloué.

```
docker ps                # repérez mongo-ipssi et mongo-express-ipssi
docker stop mongo-ipssi mongo-express-ipssi
```

0.2 Démarrer les trois nœuds (sans les assembler)

```
docker compose -f docker-compose.rs.yml up -d
docker compose -f docker-compose.rs.yml ps    # 3 conteneurs "running"
```

Les trois mongod tournent, mais **ils ne se connaissent pas encore** : ils ont été lancés avec `--replSet rs0` sans jamais recevoir de configuration. Observez cet état intermédiaire **maintenant**, il disparaîtra dans deux minutes :

```
docker exec mongo1 mongosh --quiet --eval 'printjson(db.hello())'
docker exec mongo1 mongosh --quiet --eval 'db.test.insertOne({ a: 1 })'
```

- **Q1.** Que vaut `isWritablePrimary` ? Le champ `primary` est-il présent ? Recopiez la valeur du champ `info`. Puis recopiez le `codeName` exact de l'erreur d'écriture. Formulez la conclusion : un mongod lancé avec `--replSet` mais non initialisé est-il primary, secondary, ou ni l'un ni l'autre ?

0.3 Initialiser le set

```
docker exec -i mongo1 mongosh < init-rs.js
```

Attendez ~10 secondes, puis :

```
docker exec mongo1 mongosh --quiet --eval 'rs.status().members.map(m => m.name + " " + m.stateStr).join(" | ")'
```

- **Q2.** Recopiez cette sortie. Quel nœud est **PRIMARY** ? Ouvrez `init-rs.js` : quel champ explique ce choix, et quelle est sa valeur ?

0.4 Charger les données réelles

```
curl -L -o zips.json \
  https://raw.githubusercontent.com/neelabalan/mongodb-sample-dataset/main/sample_training/zips.json
wc -l zips.json          # attendu : 29470

docker cp zips.json mongo1:/tmp/zips.json
docker exec mongo1 mongoimport --db census --collection zips --drop --file /tmp/zips.json
```

Point de contrôle P0 — `mongoimport` doit annoncer **29470 document(s) imported successfully**. L'import s'adresse au **primary** : c'est la seule règle absolue de la réplication MongoDB, **toutes les écritures passent par le primary**.

- **Q3.** Vérifiez le contenu depuis le primary. Donnez : le nombre de documents, le nombre d'**États distincts** (`distinct("state").length`), et la **population totale** (`$sum` sur `pop`). Le nombre d'États vous surprend-il ? Expliquez-le.
- **Q4. Qualité de données.** Le champ `zip` ressemble à une clé naturelle. Prouvez-le ou réfutez-le : combien de valeurs `zip` **distinctes** pour 29 470 documents ? Écrivez l'agrégation qui **liste les codes postaux apparaissant plus d'une fois** (donnez-les). Conclusion : peut-on créer un index **unique** sur `zip` ? Tentez-le réellement et recopiez l'erreur.
- **Q5.** Combien de documents ont une population `pop` **égale à 0** ? Un code postal réellement peuplé de 0 habitant, est-ce une erreur de saisie ou une réalité métier ? Argumentez en une phrase.

Partie 1 — Anatomie du Replica Set et de l'oplog (≈ 40 min)

1.1 Lire la configuration et l'état

```
docker exec mongo1 mongosh --quiet --eval 'printjson(rs.conf().members)'
```

```
docker exec mongo1 mongosh --quiet --eval 'printjson(rs.conf().settings)'
```

- **Q6.** Dans `rs.conf().settings`, relevez la valeur de `electionTimeoutMillis` et de `heartbeatIntervalMillis`. Traduisez-les en français : « un secondary déclare le primary mort au bout de ... alors qu'il l'interroge toutes les ... ». **Notez ces deux valeurs à part : vous les confronterez à un chronomètre en Q21 et vous les modifierez en R3.**
- **Q7.** Dans `rs.status()`, relevez pour chaque membre : `stateStr`, `health`, et `lastHeartbeat`. Quel champ vous dirait, en production, qu'un nœud est **injoignable** ?

1.2 L'oplog

L'oplog est une **collection capped** (taille fixe, circulaire) de la base système `local`. C'est le journal que les secondaries rejouent.

```
docker exec mongo1 mongosh --quiet --eval '
const l = db.getSiblingDB("local");
print("maxSize:", l.oplog.rs.stats().maxSize);
print("total:", l.oplog.rs.countDocuments({}));
'
```

- **Q8.** Quelle est la **taille maximale** de l'oplog en octets, et pourquoi cette valeur précise ? (Indice : regardez la ligne `command:` de `docker-compose.rs.yml`.) Que se passerait-il si vous ne la fixiez pas ?
- **Q9.** Comptez les entrées d'oplog correspondant à l'**insertion** de documents dans `census.zips` :

```
db.getSiblingDB("local").oplog.rs.countDocuments({ op: "i", ns: "census.zips" })
```

Comparez ce nombre au nombre de documents importés. **Que démontre cette égalité** sur la granularité de la réplication ? (Un `mongoimport` envoie des lots de plusieurs milliers de documents : l'oplog, lui, contient-il des lots ou des opérations unitaires ?)

- **Q10.** Affichez **une** entrée d'oplog d'insertion (`findOne({op:"i", ns:"census.zips"})`). Identifiez les champs `op`, `ns`, `o`, `ts`, `wall`. En quoi le contenu du champ `o` rend-il l'opération **idempotente** (rejouable 2 fois sans dommage) ?
- **Q11. Preuve par l'expérience.** Lancez sur le primary un `updateMany` qui incrémente la population de tous les codes postaux du Texas :

```
db.zips.updateMany({ state: "TX" }, { $inc: { pop: 1 } })
```

Puis lisez **une** entrée d'oplog correspondante (`findOne({ op: "u", ns: "census.zips" })`) et regardez le champ `o`. **Y trouvez-vous un `$inc`** ? Que trouvez-vous à la place, et pourquoi MongoDB procède-t-il ainsi ? Reliez votre réponse à l'idempotence de la Q10.

- **Q12. Dimensionnement — à calculer, pas à réciter.** Relevez `size` et `count` de `db.getSiblingDB("local").oplog.rs.stats()`.
 - **(a)** Calculez la **taille moyenne d'une opération** sur votre oplog (`size / count`). Donnez la valeur.
 - **(b)** Déduisez-en **combien d'opérations** cet oplog de 128 Mo peut mémoriser avant d'écraser les plus anciennes.
 - **(c)** Votre production encaisse **300 écritures par seconde**. Convertissez votre résultat (b) en **fenêtre de réplication**, exprimée en heures. Un secondary tombe le vendredi à 18 h : peut-il encore rattraper le lundi à 9 h par l'oplog ? Que se passe-t-il sinon ?

Partie 2 — Lire et écrire dans un Replica Set (≈ 30 min)

- **Q13.** Connectez-vous **directement sur mongo2** (un secondary) et comptez les documents :

```
docker exec mongo2 mongosh --quiet census --eval 'db.zips.countDocuments({})'
```

Obtenez-vous les données ? Le cours parlait de `rs.secondaryOk()` : cherchez dans la documentation ce que **mongosh** positionne automatiquement lorsqu'on se connecte directement à un membre, et expliquez pourquoi l'ancienne commande n'est plus nécessaire.

- **Q14.** Toujours sur **mongo2**, tentez une **écriture** :

```
docker exec mongo2 mongosh --quiet census --eval 'db.zips.insertOne({ test: 1 })'
```

Recopiez le **nom exact de l'erreur** (`codeName`) et son message. Pourquoi MongoDB refuse-t-il, alors qu'il vous a laissé lire ?

- **Q15.** Mesurez le **retard de réplication** : `rs.printSecondaryReplicationInfo()` depuis le primary. Que valent les retards ? Écrivez maintenant 1 000 documents d'un coup dans une collection `census.charge` puis relancez la commande : le retard bouge-t-il ? Concluez sur le caractère **asynchrone** de la réplication.
- **Q16. Read Preference.** Depuis mongo1, exécutez la même requête avec deux préférences différentes :

```
db.getMongo().setReadPref("primary"); db.zips.countDocuments({ state: "NY" })
db.getMongo().setReadPref("secondary"); db.zips.countDocuments({ state: "NY" })
```

Le résultat est-il identique ? Donnez **un cas métier** où lire sur un secondary est acceptable, et **un cas** où c'est dangereux (pensez au mot *stale*).

Partie 3 — Failover : provoquer la panne et la chronométrer (≈ 50 min)

C'est le cœur de la journée. Vous allez tuer le primary **deux fois, de deux manières différentes**, et mesurer. **Les chiffres que vous obtiendrez sont les vôtres** — ils dépendent de votre machine, de la charge et de l'instant. C'est ce qui rend cette partie non recopiable.

3.1 Outil de mesure

Le fichier `watch_primary.py` vous est fourni : il interroge le cluster toutes les 300 ms et affiche, avec un horodatage relatif, **chaque changement de primary**.

```
python watch_primary.py          # observe via mongo2
python watch_primary.py mongo3   # si mongo2 est le primary courant
```

Laissez-le tourner dans un terminal, et exécutez les pannes dans un autre.

3.2 Panne « propre » (arrêt maîtrisé)

```
docker stop mongo1      # SIGTERM : mongod a le temps de prévenir le set
```

- **Q17.** Combien de temps s'écoule entre l'arrêt et l'apparition d'un **nouveau primary** ? Quel nœud est élu ? Notez la valeur dans `failover.md`.
- **Q18.** Pendant la bascule, `rs.status()` depuis mongo2 : quel est le `stateStr` et le `health` de mongo1 ?

3.3 Retour du nœud et reprise de rôle

```
docker start mongo1
```

- **Q19.** Dans quel état mongo1 revient-il **immédiatement** ? Attendez, puis observez : redevient-il PRIMARY ? **Au bout de combien de temps ?** Expliquez le mécanisme à l'aide de `rs.conf().members[0].priority` (c'est un *priority takeover*). **Combien de bascules le cluster a-t-il subies au total depuis le `docker stop` ?** En quoi est-ce un argument contre les priorités asymétriques en production ?
- **Q20.** Comment mongo1 récupère-t-il les écritures faites pendant son absence ? Prouvez-le : avant de le redémarrer, insérez des documents sur le nouveau primary, puis vérifiez leur présence sur mongo1 après retour **en vous connectant directement à lui**. Quel mécanisme (vu en Partie 1) a servi ?

3.4 Panne brutale

Une fois mongo1 redevenu PRIMARY, recommencez avec une coupure franche :

```
docker kill mongo1      # SIGKILL : aucun préavis, comme une alim débranchée
```

- **Q21. La question centrale de la journée.** Mesurez à nouveau le délai d'élection. **Il est très différent du 3.2 : de combien exactement ?** Donnez les deux valeurs mesurées et leur rapport. Puis reliez votre mesure de panne brutale à la valeur d'`electionTimeoutMillis` relevée en Q6 : votre délai est-il **égal**, **supérieur** ou **légèrement inférieur** à ce timeout ? Expliquez l'écart constaté (*indice : à quel instant précis le compte à rebours démarre-t-il ? Regardez `heartbeatIntervalMillis`*).
- **Q22. Synthèse.** Complétez ce tableau dans `failover.md` et commentez en 3 lignes ce que vous annonceriez à la DSI sur le SLA (rappel : 99,9 % = 43 min/mois) :

Scénario	Commande	Délai mesuré	Nœud élu	Écritures perdues ?
Arrêt propre	<code>docker stop</code>			
Panne brutale	<code>docker kill</code>			
Retour du nœud	<code>docker start</code>	(délai de reprise)		

- **Q23. Le quorum.** Redémarrez mongo1, puis arrêtez **deux** nœuds sur trois (`docker stop mongo2 mongo3`). Interrogez le nœud survivant **immédiatement**, puis **de nouveau 15 secondes plus tard** :

```
docker exec mongo1 mongosh --quiet --eval 'print(db.hello().isWritablePrimary, rs.status().myState)'
```

- (a) Les deux relevés **diffèrent**. Donnez-les et expliquez ce qui s'est passé entre les deux.
- (b) Le nœud survivant peut-il encore accepter une **écriture** ? Une **lecture** ? Testez les deux et recopiez les erreurs.
- (c) **Expliquez avec le mot « majorité »** pourquoi un Replica Set de 3 nœuds tolère 1 panne et pas 2, et pourquoi un set de 4 nœuds ne tolère **pas** mieux qu'un set de 3.

Partie 4 — Write Concern & Read Concern (≈ 35 min)

Remettez les 3 nœuds en marche (`docker start mongo1 mongo2 mongo3`) et vérifiez `rs.status()`.

- **Q24.** Insérez un document avec `{ writeConcern: { w: 1 } }` puis un autre avec `{ writeConcern: { w: "majority" } }` dans `census.demo`. Les deux réussissent. Quelle est la **différence de garantie** ? Dans quel scénario précis de la Partie 3 le `w: 1` aurait-il pu **perdre** l'écriture ?
- **Q25.** Tentez maintenant :

```
db.demo.insertOne({ a: 1 }, { writeConcern: { w: 4, wtimeout: 3000 } })
```

Recopiez le `codeName` et le message exacts. **Chronométrez la commande**. Pourquoi MongoDB refuse-t-il **immédiatement** au lieu d'attendre les 3 secondes de `wtimeout` ?

- **Q26. La question d'écart de la journée.** Arrêtez un nœud secondary (`docker stop mongo3`) puis, depuis le primary :

```
db.demo.insertOne({ b: 1 }, { writeConcern: { w: "majority", wtimeout: 3000 } })
db.demo.insertOne({ c: 1 }, { writeConcern: { w: 3, wtimeout: 3000 } })
```

- (a) L'une passe, l'autre échoue. Laquelle, avec quel `codeName` ?
- (b) Comptez maintenant les documents : `db.demo.countDocuments({})`. **Combien en trouvez-vous ?** Combien en attendiez-vous si un échec signifiait « rien n'a été écrit » ? **Donnez l'écart.**
- (c) Expliquez ce résultat contre-intuitif : que signifie exactement l'échec d'un write concern ? Quelle conséquence cela a-t-il pour une application qui **rejouerait** l'écriture après l'erreur ?
- **Q27.** Ajoutez `j: true` à un write concern. Que garantit ce paramètre de plus, et à quel coût ? Reliez-le à la question : « *que se passe-t-il si les 3 machines perdent le courant en même temps ?* »
- **Q28. Read Concern.** Expliquez en 3 lignes ce que change `readConcern: "majority"` par rapport à `"local"` du point de vue d'un utilisateur final, en réutilisant votre observation de Q26 (une écriture peut exister localement sans être confirmée par la majorité).

Partie 5 — Résilience applicative : ce que voit vraiment l'application (≈ 45 min)

Jusqu'ici vous avez mesuré le failover **vu du cluster**. La DSI, elle, se moque de `rs.status()` : elle veut savoir ce que **l'utilisateur** perd. Vous allez brancher une vraie application Python sur le set et la faire tourner pendant une panne.

Redémarrez les trois nœuds si besoin (`docker start mongo1 mongo2 mongo3`).

5.1 Le piège de l'URI

Le fichier `writer.py` vous est fourni **partiellement complété** : il insère un document par seconde dans `census.heartbeat` et journalise, ligne par ligne, l'horodatage, le primary courant et le résultat de l'écriture.

Première tentative, **depuis votre poste** :

```
pip install "pymongo>=4.6"
python writer.py "mongodb://localhost:27017,localhost:27018,localhost:27019/?replicaSet=rs0"
```

- **Q29. Cela échoue** en `ServerSelectionTimeoutError`. Recopiez le message **en entier**, y compris la `Topology Description`.
 - (a) Quels **noms d'hôtes** le driver dit-il avoir essayé de joindre ?
 - (b) Vous avez pourtant écrit `localhost` trois fois. **D'où sortent ces noms ?** (Indice : relisez `init-rs.js` — que contient `rs.conf()` ? Que fait un driver quand on lui précise `?replicaSet=` ?)
 - (c) Vérifiez votre hypothèse : relancez **sans** `?replicaSet=rs0`, sur `mongodb://localhost:27017` seul. **Cela échoue encore**. Votre hypothèse du (b) était-elle donc fausse ? Regardez ce que le nœud annonce de lui-même :

```
docker exec mongo1 mongosh --quiet --eval 'const h=db.hello(); print(h.setName); printjson(h.hosts)'
```

Reformulez : ce n'est pas le paramètre `?replicaSet=` qui déclenche le remplacement de la liste, c'est **la découverte du set elle-même**, que le driver fait dès qu'un nœud se déclare membre d'un Replica Set.

- (d) Trouvez dans la documentation PyMongo l'option d'URI qui **désactive** cette découverte et force une connexion à une machine unique. Relancez avec. Ça marche — mais affichez `client.topology_description.topology_type_name` et `client.primary`. **Que valent-ils, et qu'avez-vous perdu au passage ?**

5.2 Lancer l'application dans le réseau du cluster

```
docker run --rm --network rslab_default -v "$PWD:/app" -w /app python:3.12-slim \
sh -c "pip install -q 'pymongo>=4.6' && python writer.py 'mongodb://mongo1:27017,mongo2:27017,mongo3:27017/?replicaSet=rs0&retryWrites=true'"
```

- **Q30.** Le script tourne et écrit une ligne par seconde. Recopiez les 5 premières lignes dans `resilience.md`. Quel primary voit-il ?

5.3 Tuer le primary pendant que l'application écrit

Dans un **autre terminal**, pendant que `writer.py` tourne :

```
docker kill mongo1
```

- **Q31. La mesure qui compte.** Laissez le script aller à son terme, puis collez **sa sortie brute et horodatée** dans `resilience.md`.
 - **(a)** Combien de **secondes consécutives** d'écritures ont échoué ? Donnez l'horodatage de la première ligne en échec et de la première ligne redevenue OK.
 - **(b)** Combien d'écritures **réussies** et **échouées** au total (le script les compte) ?
 - **(c)** Le driver s'est-il reconnecté **seul**, sans que vous ne redémarriez quoi que ce soit ? À quelle ligne voyez-vous le changement de primary ?
 - **(d)** Comparez cette durée d'indisponibilité à votre mesure de la Q21. Sont-elles égales ? Si elles diffèrent, proposez une explication.

- **Q32. `retryWrites` : l'expérience qui ne prouve rien, puis celle qui prouve.**

(a) Première tentative. Relancez exactement le même scénario, mais avec `retryWrites=false` dans l'URI, et tuez à nouveau le primary. Donnez les deux décomptes d'échecs (avec et sans `retryWrites`). **Quel est l'écart ?**

(b) Vous devriez trouver un écart **nul ou quasi nul**, et c'est le résultat attendu — ne cherchez pas l'erreur. Regardez le **type d'exception** levé dans les deux cas : c'est le même. Formulez l'explication : pendant une bascule, combien de temps y a-t-il **zéro primary** dans le cluster (votre Q21) ? Que fait le driver pendant ce temps — il échoue immédiatement, ou il **attend** ? (Regardez la durée affichée sur la ligne en échec et comparez-la à `serverSelectionTimeoutMS`, fixé à 5000 dans le script.) `retryWrites` rejoue **une fois** : cela peut-il aider quand il n'y a aucun primary à qui parler ?

(c) L'expérience qui prouve. Il faut une panne d'un autre genre : un primary qui **rérograde en restant vivant**. C'est ce que fait `rs.stepDown()`. Relancez le script — d'abord avec `retryWrites=true`, puis avec `retryWrites=false` — et, au lieu de tuer le nœud, exécutez depuis le primary :

```
docker exec <primary> mongosh --quiet --eval 'rs.stepDown(20)'
```

- Donnez les deux résultats. **Cette fois l'écart est net.**
- Avec `retryWrites=false`, recopiez le **type d'exception** et le **code d'erreur** exacts. En quoi diffèrent-ils de ceux du (a) ?
- Concluez en une phrase : **contre quel type de panne `retryWrites` protège-t-il réellement**, et contre lequel ne peut-il rien ?

(d) Cherchez à quelle condition `retryWrites` peut rejouer une écriture **sans risque de doublon**. (Indice : vous avez vu les champs responsables en Q10, dans l'entrée d'oplog d'une insertion.) Pourquoi `updateMany` et `deleteMany` ne sont-ils, eux, **jamais** rejoués automatiquement ?

- **Q33. Le décompte final.** Le script affiche en fin d'exécution le nombre d'écritures qu'il **croit** avoir réussies, puis le `count_documents` réel de la collection.
 - (a) Les deux nombres coïncident-ils ? Donnez-les et **calculez l'écart**.
 - (b) Rejouez le scénario en forçant `w: "majority"` sur les insertions du script. L'écart change-t-il ?
 - (c) Rédigez, en deux phrases, **le chiffre que vous annoncez à la DSI** : « lors d'une panne serveur brutale, notre service est indisponible en écriture pendant ... et perd au plus ... écriture(s), à condition que ... ».

Partie 6 — Réflexion (≈ 25 min) — dans `reponses_jour3.md`

Ces quatre questions ne se répondent pas de mémoire : **chacune exige que vous repartiez de vos propres mesures et, pour trois d'entre elles, que vous refassiez une manipulation**. Une réponse qui ne cite aucun de vos chiffres ne sera pas prise en compte.

- **R1. Le collègue qui veut un 4^e nœud.** Un collègue propose de passer le Replica Set à 4 nœuds « pour plus de sécurité ». **Vérifiez-le expérimentalement** avant de répondre :

```
docker run -d --name mongo4 --network rslab_default mongo:7.0 \
  mongod --replSet rs0 --bind_ip_all --port 27017 --oplogSize 128
docker exec mongo1 mongosh --quiet --eval 'rs.add("mongo4:27017")'
```

Attendez la synchronisation, puis **arrêtez deux nœuds** et interrogez les survivants. Le set accepte-t-il encore les écritures avec 4 membres et 2 pannes ? Et avec 3 membres et 1 panne (votre Q23) ? Donnez les deux résultats observés, puis répondez au collègue en une phrase. **Que proposeriez-vous à la place si le budget ne permet que 4 machines ?** (Nettoyage : `rs.remove("mongo4:27017")` puis `docker rm -f mongo4`.)

- **R2. Deux problèmes, deux réponses.** Réplication et sharding répondent à **deux problèmes différents** ; vous shardez demain. Formulez chacun des deux problèmes en une phrase. Puis, en repartant de votre **Q23(c)** sur la majorité, calculez le **nombre de machines** d'un cluster de production à 3 shards (config servers + mongos + nœuds par shard) et expliquez pourquoi un cluster shardé dont les shards ne seraient **pas** répliqués serait **plus fragile** qu'un simple Replica Set.
- **R3. Régler le curseur — expérience obligatoire.** Vous avez relevé `electionTimeoutMillis = 10000` en Q6 et mesuré un délai de bascule en Q21. Descendez ce paramètre à **2000 ms** :

```
cfg = rs.conf(); cfg.settings.electionTimeoutMillis = 2000; rs.reconfig(cfg)
```

Refaites un `docker kill` du primary, `watch_primary.py` en marche, et **mesurez à nouveau**.

- **(a)** Donnez les deux délais mesurés (10 000 ms et 2 000 ms) et leur rapport. La bascule est-elle **5 fois plus rapide** ? Sinon, quelle part du délai **ne dépend pas** de ce paramètre ?
 - **(b)** Quel **risque** prend-on à descendre ce timeout trop bas ? Pensez à un réseau qui a un hoquet de 3 secondes, et à ce que coûte une élection.
 - **(c)** Remettez la valeur d'origine. Quelle valeur recommanderiez-vous à la DSI, et sur quel argument chiffré ?
- **R4. Le chiffre honnête.** Vous devez livrer à la DSI une phrase unique sur la tenue du SLA. Croisez **votre Q21** (délai vu du cluster), **votre Q31** (indisponibilité vue de l'application) et **votre Q26** (une écriture peut exister sans être confirmée). Rédigez cette phrase, puis expliquez en 3 lignes **pourquoi annoncer le seul chiffre de la Q21 serait malhonnête**.

Pour aller plus loin (facultatif)

À traiter une fois les Parties 0 à 6 rendues. Chacun de ces points suppose **une mesure**, pas une intention.

- **B1. L'arbitre et le faux sentiment de sécurité.** Ajoutez un arbitre (`rs.addArb()`), regardez `rs.status()`. Quelles données stocke-t-il ? Puis **montez le piège** : sur un set `Primary + Secondary + Arbitre`, arrêtez le secondary et tentez une écriture en `w: "majority"` avec un `wtimeout`. Que se passe-t-il, alors que le primary est toujours élu et se déclare en bonne santé ? Vous venez de reproduire la raison exacte pour laquelle MongoDB déconseille les arbitres.
- **B2. Le membre caché et le membre retardé.** Reconfigurez mongo3 en `hidden: true, priority: 0`, puis en `secondaryDelaySecs: 60`. Vérifiez le retard réel en insérant un document daté et en le cherchant sur mongo3 toutes les 10 secondes. À quoi sert concrètement un membre retardé ? (*Indice : quelle catastrophe un backup ne rattrape-t-il pas assez vite ?*)
- **B3. Authentifier le Replica Set.** Le compose fourni **n'active pas l'authentification** : un Replica Set authentifié exige un `keyFile` partagé entre les nœuds. Générez-en un (`openssl rand -base64 756`), montez-le dans les 3 conteneurs avec les bons droits (`chmod 400`, propriétaire `mongodb`), relancez le set avec `--keyFile` et `--auth`, et recréez l'utilisateur admin. Documentez les deux erreurs que vous rencontrerez en chemin — c'est l'opération d'exploitation la plus classique et la plus casse-figure du métier.
- **B4. Le rollback, en vrai.** Le cours affirme qu'une écriture confirmée en `w: 1` peut être **annulée** après un failover. Prouvez-le : isolez le primary du reste du set (`docker network disconnect`), écrivez-y en `w: 1`, laissez les deux autres élire un nouveau primary, puis reconnectez l'ancien. Cherchez le fichier de rollback produit dans son répertoire de données et vérifiez que le document a **disparu** de la collection. C'est la manipulation la plus spectaculaire du module.

Checklist finale avant rendu

`reponses_jour3.md` : Q1→Q33 avec commande + sortie observée, R1→R4 rédigées **en citant vos chiffres**

`failover.md` : tableau des 3 scénarios avec délais **mesurés**

`resilience.md` : sortie brute horodatée de `writer.py`, décompte des écritures perdues, comparaison avec/sans `retryWrites`

`writer.py` complété

Les conteneurs sont arrêtés proprement en fin de séance :

```
docker compose -f docker-compose.rs.yml down -v
```

Le port 27017 est libéré pour le Jour 4